

Esercizio 2 Aspirapolvere

domenica 8 marzo 2026 10:27

Un processore z64 controlla un ROBOT ASPIRAPOLVERE. Il robot è costituito da una periferica SPAZZOLA, due periferiche MOTORE_1 e MOTORE_2, uno posto a destra ed uno a sinistra della struttura del robot, che ne consentono il movimento. In aggiunta, sono presenti due periferiche INTERRUETTORE_1 e INTERRUETTORE_2, che consentono di percepire se il robot ha urtato una parete della stanza o un ostacolo. E' presente in aggiunta una periferica TIMER. Uno schema del robot è riportato nella seguente figura. Entrambe le periferiche MOTORE hanno a disposizione un registro (di dimensione byte) il cui valore determina il movimento. Se il registro contiene il valore 0, il motore non si muove, se contiene il valore 1 si muove in avanti, se contiene il valore -1 si muove all'indietro. All'avvio del sistema, lo z64 programma la periferica SPAZZOLA per incominciare a raccogliere la polvere, e in aggiunta programma MOTORE_1 e MOTORE_2 per muoversi in avanti. Quando il robot incontra un ostacolo una delle due periferiche INTERRUETTORE invia un'interruzione e il protocollo per aggirare l'ostacolo è il seguente:

- La periferica SPAZZOLA viene fermata
- Se l'impatto è rilevato da INTERRUETTORE_1, viene programmato MOTORE_1 per muoversi in avanti e MOTORE_2 per muoversi all'indietro.
- Se l'impatto è rilevato da INTERRUETTORE_2, viene programmato MOTORE_2 per muoversi in avanti e MOTORE_1 per muoversi all'indietro.
- Viene programmata la periferica TIMER per inviare un'interruzione dopo 4 secondi (valore già presente nella periferica)
- Allo scadere dei 4 secondi, le periferiche MOTORE vengono programmate per muoversi in avanti, e SPAZZOLA viene avviata.

Progettare:

- L'interfaccia di una periferica MOTORE, di una periferica INTERRUETTORE e di TIMER
- Il codice di inizializzazione del robot, e il driver di INTERRUETTORE e TIMER.

Soluzione : **si presuppone spazzola in busy waiting**

```
.org 0x800
.data:
.equ $mot1_status,0x01
.equ $mot2_status,0x02
.equ $mot1_reg,0x03
.equ $mot2_reg,0x04
.equ $interruttore_reg,0x05
.equ $timer_reg,0x06
.equ $interruttore_irq,0x07
.timer : .byte 0 #contatore del tempo -> reg

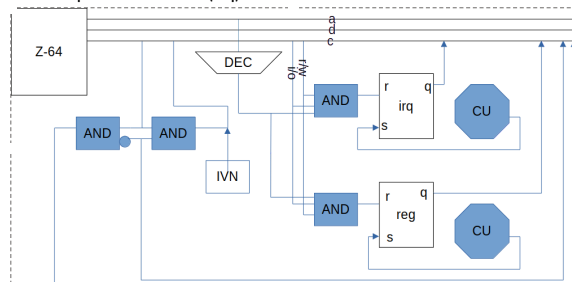
.text
Main: #spazzola
    Sti
    .loop:
        Jmp .motore1
    L1:
        Jmp .motore2
    L2:
        Xori %al,%al
        Addb $4,%al
        Outb %al,$timer_reg
        Movb %al,timer
        Hlt
        Jmp .loop
.motore1:
    Outb %al,$mot1_status #a%=0
    .bw:
        Inb $mot1_status,%al
        Btb $0,%al
        Jnc .bw
    #dato pronto
    Movb $1,%al
    Outb %al,$interruttore_reg # interruttore_reg =1 -> interruzione dal motore dx
    Jmp L1
.motore2:
    Xorb %al,%al #azzerato al
    Outb %al,$mot2_status
    .bw1:
        Inb $reg2_status,%al
        Btb $0,%al
        Jnc .bw1
```

```

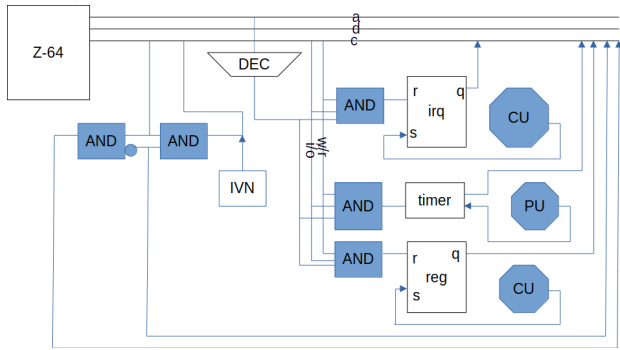
        #dato pronto
        Movb $1,%al
        Outb %al,$mot2_reg
        Movb $2,%al
        Outb %al,$interruttore_reg # interruttore_reg =2-> interruzione dal motore sx
        Jmp L2
.driver0 :#interruttore_1 --dx
        Pushq %rax
        Outb %al,interruttore_irq
        Call funct1
        Popq %rax
        Iret
Func1:
#fermo spazzola
Outb %al,$mot1_reg #reg=0
Outb %al,$mot2_reg #reg=0
#setto motore
Movb $1,%al
Outb %al,$mot1_reg
Xorb %al,%al #azzerò al
#-1 indietro
Addb $1,%al
Neg %al #negativo -> cp a 2
Outb %al,$mot2_reg
ret
.driver1 : #interruttore_2--sx
        Pushq %rax
        Outb %al,interruttore_irq
        Call func2
        Popq %rax
        Iret
Func2:
#fermo spazzola
Outb %al,$mot1_reg #reg=0
Outb %al,$mot2_reg #reg=0
#setto motore
Movb $1,%al
Outb %al,$mot2_reg
Xorb %al,%al #azzerò al
#-1 indietro
Addb $1,%al
Neg %al #negativo -> cp a 2
Outb %al,$mot1_reg
ret
.driver2: #timer
        Pushw %rax
        Call func3
Func3:
        Cmpb $4,timer # timer è uguale a 4
        Jnz .increment
        Jz .avvia
.increment:
        Addb $1,timer
        Cmpb $4,timer
        Jnz .increment
        Jz .avvia
.avvia:
        Movb $1,%al
        Outb %al,$mot1_reg
        Outb %al,$mot2_reg
        Popw %rax
        iret

```

Schema per interruttore (irq)



Schema per timer (irq)



Schema per spazzola (bw) ; notare che i due flip flop andrebbero duplicati in quanto doppia spazzola

